PRACTICAL ACTIVITY 1: AES Encryption and Decryption Using a Crypto Library

Objective:

To implement symmetric encryption and decryption using AES, demonstrating secure handling of data with a standard cryptographic library.

Tools & Setup:

LANGUAGE: Python

LIBRARY: cryptography

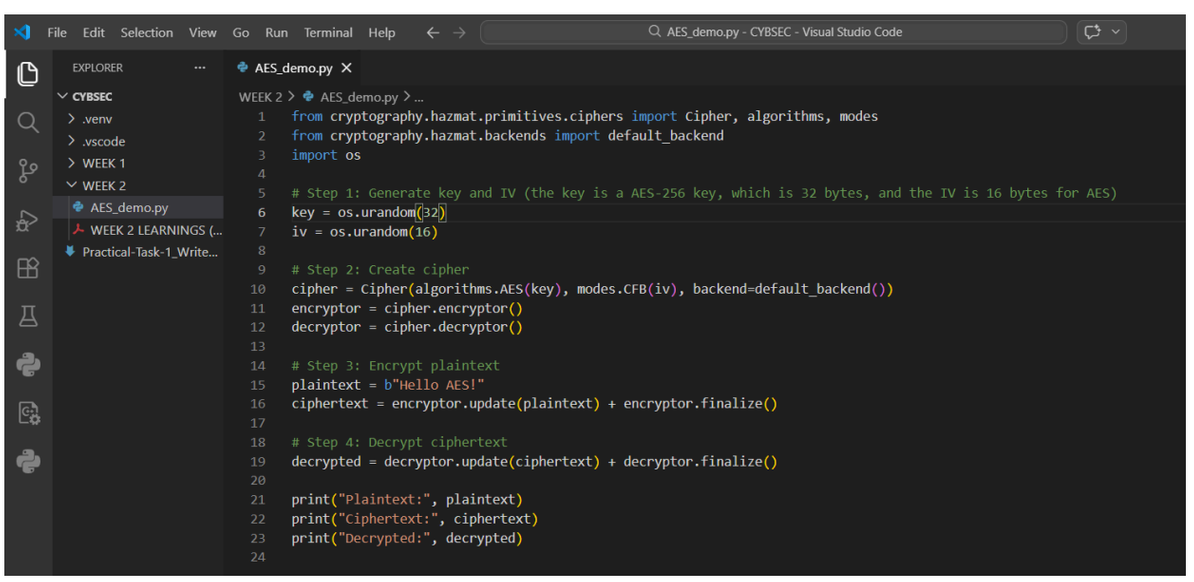
ENVIRONMENT: Local machine, test script

PROCEDURE:

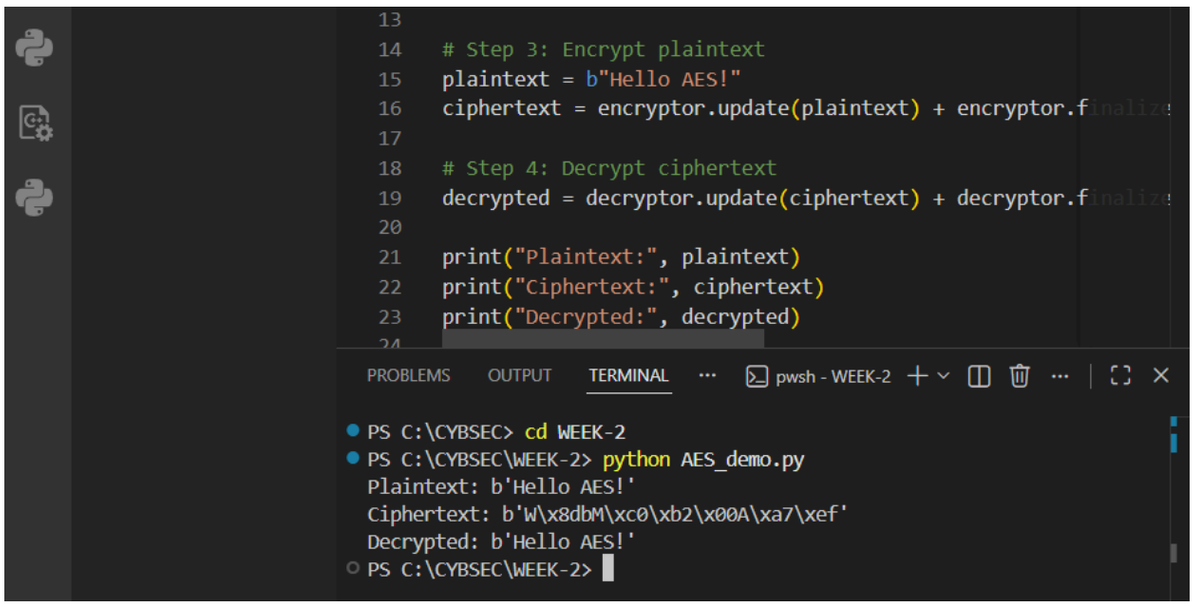
To complete the activity, I began by preparing my environment. I opened my terminal and ensured that Python was installed and ready to use. Once the environment was set, I proceeded to install the required cryptography library by entering the command `pip install cryptography`. The successful installation confirmed that I had the necessary tools to carry out AES encryption and decryption.

```
C:\Users\jurer>pip install cryptography
Defaulting to user installation because normal site-packages is not writeable
Collecting cryptography
  Downloading cryptography-46.0.5-cp311-abi3-win_amd64.whl.metadata (5.7 kB)
Collecting cffi>=2.0.0 (from cryptography)
  Downloading cffi-2.0.0-cp312-cp312-win_amd64.whl.metadata (2.6 kB)
Collecting pycparser (from cffi>=2.0.0->cryptography)
  Downloading pycparser-3.0-py3-none-any.whl.metadata (8.2 kB)
Downloading cryptography-46.0.5-cp311-abi3-win_amd64.whl (3.5 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.5/3.5 MB 3.0 MB/s  0:00:01
Downloading cffi-2.0.0-cp312-cp312-win_amd64.whl (183 kB)
Downloading pycparser-3.0-py3-none-any.whl (48 kB)
Installing collected packages: pycparser, cffi, cryptography
Successfully installed cffi-2.0.0 cryptography-46.0.5 pycparser-3.0
```

Next, I created a new Python file named `AES_demo.py` to serve as the workspace for my implementation. Within this file, I imported the relevant modules from the `cryptography` package and the `os` library, which allowed me to generate secure random values for the encryption key and initialization vector (IV).



I then generated a 256-bit AES key and a 16-byte IV. With these values prepared, I constructed a cipher object using AES in CFB mode. This cipher object provided both an encryptor and a decryptor, enabling me to perform the two essential operations of this activity. With the cipher ready, I defined a simple plaintext message: `"Hello AES!"`. Using the encryptor, I transformed this plaintext into ciphertext, which appeared as unreadable binary data. Immediately after, I applied the decryptor to the ciphertext, successfully recovering the original plaintext.



Finally, I verified the results. The decrypted message matched the original input exactly, confirming that the encryption and decryption process was implemented correctly. This validation step demonstrated the reliability of AES when used with proper key and IV management.

Reflection:

Completing this activity gave me a clearer understanding of how symmetric encryption works in practice. By using AES, I saw firsthand how plaintext can be transformed into ciphertext that appears completely random, and then successfully restored through decryption when the correct key and initialization vector are applied. This reinforced the importance of proper key management, since without the right key the data remains inaccessible.

One challenge I encountered was ensuring that the encryption and decryption processes were correctly paired with the same parameters. Once I resolved this, the workflow became straightforward, and it highlighted how cryptography libraries simplify complex mathematical operations while still requiring careful attention to detail.

This exercise demonstrated the reliability of AES and its relevance in securing real-world applications such as file storage and communication systems. It also emphasized that while libraries make implementation easier, the responsibility for secure usage lies in how developers manage keys, modes, and data.